# Python Study Guide

# Chapter 18 — File Operations in Python (Text and Binary Files)

## 1. Chapter title

**File Operations in Python (Text and Binary Files)**

---

## 2. Concept explanation

File handling is one of the most practical topics in Python because real programs often need to **store**, **read**, **update**, or **transfer** data using files.

A file may contain:

- plain text
- numbers
- logs
- CSV-style data
- program output
- images or other binary content

Python makes file handling straightforward through the built-in `open()` function.

### Why file handling matters

Without file operations, your data exists only while the program is running.

Once the program stops, the data disappears from memory.

Files allow you to:

- save results permanently
- read data from earlier work
- process reports and logs
- exchange data between programs
- copy or back up content
- work with text and binary files

## Basic file syntax

A common and recommended way to work with files is:

```
with open("filename.txt", "r") as file:
    data = file.read()
```

This is called a **context manager**.

It is preferred because:

- the file opens safely
- Python closes it automatically
- the code is cleaner
- it reduces file-handling mistakes

## File modes

The mode tells Python what you want to do with the file.

Common modes:

- `r` -> read
- `w` -> write
- `a` -> append

- `rb` -> read binary
- `wb` -> write binary
- `w+` -> write and read

#### Meaning of the most important modes

**`r` mode**

- reads an existing file
- raises an error if the file does not exist

**`w` mode**

- creates a new file if needed
- overwrites existing content if the file already exists

**`a` mode**

- adds new content at the end of the file
- keeps the existing content safe

### Reading files

You can read a file in different ways:

- `read()` -> reads the whole file
- `readline()` -> reads one line
- `readlines()` -> reads all lines into a list
- loop over the file -> reads line by line

### Writing files

You can write content using:

- `write()` -> write a single string
- `writelines()` -> write multiple strings from a list

### Appending files

Appending is useful when you want to add content without deleting what is already there.

This is common in:

- logs
- progress notes
- result tracking
- report updates

### Text files vs binary files

A **text file** stores readable characters.

Examples:

- `.txt`
- `.csv`
- `.py`
- `.json`

A **binary file** stores raw bytes.

Examples:

- images
- audio files
- PDFs
- compiled data

Text files are handled as strings.

Binary files are handled as bytes.

### `w+` mode and `seek()`

The `w+` mode allows both writing and reading.

But after writing, the file pointer is often at the end of the file.

So you may need:

```
file.seek(0)
```

This moves the cursor back to the beginning so you can read what was written.

### Lecture points preserved and expanded

From your lecture notes, the key ideas are:

- files can be read using `with open`
- you can read the whole file or line by line
- file modes such as `r`, `w`, and `a` are important
- `w` mode overwrites previous data
- `a` mode appends without removing existing content
- `writelines()` can write multiple lines
- binary files use `rb` and `wb`
- assignments include counting lines, words, and characters
- copying content between files is an important exercise
- `w+` supports reading and writing together
- `seek()` controls the file cursor position

---

## 3. Why it matters

File operations matter because they move Python from simple in-memory exercises to real-world programming.

Many practical programs depend on files:

- text processors
- automation scripts
- log analyzers
- report generators
- configuration managers
- data cleaning tools
- backup utilities

Once you understand file handling well, you can build programs that:

- save user input
- process data from files
- update records
- copy and transform content
- work with images or other binary data

This chapter is especially important because it also strengthens understanding of:

- strings
- loops
- conditionals
- error prevention
- resource management

---

## 4. Code examples

### Example 1: Read an entire text file

with open("sample.txt", "r") as file:

```
    content = file.read()
    print(content)
```

### Example 2: Read a file line by line

```
with open("sample.txt", "r") as file:
    for line in file:
        print(line.strip())
```

### Example 3: Write to a file using `w` mode

```
with open("notes.txt", "w") as file:
    file.write("Python file handling is useful.\n")
    file.write("This file was created in write mode.\n")
```

### Example 4: Read back written content

```
with open("notes.txt", "r") as file:
    print(file.read())
```

### Example 5: Append content using `a` mode

```
with open("notes.txt", "a") as file:
    file.write("This line was appended later.\n")
```

### Example 6: Write multiple lines using `writelines()`

```
lines = ["Line 1\n", "Line 2\n", "Line 3\n"]
with open("multi.txt", "w") as file:
```

```
    file.writelines(lines)
```

### Example 7: Use `readlines()`

```
with open("multi.txt", "r") as file:
    all_lines = file.readlines()
    print(all_lines)
```

### Example 8: Count lines, words, and characters

```
with open("multi.txt", "r") as file:
    lines = file.readlines()

line_count = len(lines)
word_count = sum(len(line.split()) for line in lines)
char_count = sum(len(line) for line in lines)

print("Lines:", line_count)
print("Words:", word_count)
print("Characters:", char_count)
```

### Example 9: Copy content from one file to another

```
with open("notes.txt", "r") as source:
    content = source.read()

with open("copy_notes.txt", "w") as target:
    target.write(content)
```

### Example 10: Use `w+` mode

```
with open("combined.txt", "w+") as file:
    file.write("Hello from w+ mode")
    file.seek(0)
    print(file.read())
```

### Example 11: Work with binary write mode

```
data = b"Python binary data"
with open("binary_file.bin", "wb") as file:
    file.write(data)
```

### Example 12: Work with binary read mode

```
with open("binary_file.bin", "rb") as file:
    data = file.read()
    print(data)
```

### Example 13: Read one line at a time with `readline()`

```
with open("multi.txt", "r") as file:
    print(file.readline().strip())
    print(file.readline().strip())
```

### Example 14: Safe overwrite warning example

```
with open("warning_demo.txt", "w") as file:
```

```
    file.write("Old content will be replaced in w mode.\n")
```

### Example 15: Strip whitespace while reading

```
with open("multi.txt", "r") as file:
    cleaned = [line.strip() for line in file]
    print(cleaned)
```

---

## 5. Explanation of code

### Example 1 explanation

This teaches the most direct way to read an entire file using `read()`.

### Example 2 explanation

This shows a memory-friendly and readable way to process a file line by line.

The `strip()` method removes the newline at the end.

### Example 3 explanation

This demonstrates file creation and writing using `w` mode.

If the file already exists, old content is replaced.

### Example 4 explanation

This confirms that written content can be read back normally.

### Example 5 explanation

This introduces appending, which adds content without deleting the earlier text.

### Example 6 explanation

This shows how `writelines()` can write several strings at once.

You must include `\n` manually if you want line breaks.

### Example 7 explanation

This teaches that `readlines()` returns a list where each item is one line.

### Example 8 explanation

This is an important practical exercise.

It combines file handling with loops, string splitting, and counting logic.

### Example 9 explanation

This demonstrates how file copying works by reading from one file and writing into another.

### Example 10 explanation

This teaches `w+` mode and the importance of `seek(0)` after writing.

### Example 11 explanation

This introduces binary writing.

Binary data must be written as bytes, not normal strings.

### Example 12 explanation

This shows how binary files are read back as bytes.

### Example 13 explanation

This is useful when you want manual control over line-by-line reading.

### Example 14 explanation

This reinforces the danger of `w` mode: existing content can be overwritten.

### Example 15 explanation

This is a practical cleaning pattern when working with text files.

---

## 6. Common mistakes

### 1. Using `w` mode when you meant `a`

This can erase important existing content.

### 2. Forgetting that `read()` moves the cursor

If you read once and then try to read again, you may get empty output unless you reset the cursor.

### 3. Forgetting `seek(0)` in `w+` mode

After writing, the cursor is at the end of the file.

Reading immediately without moving it back may return nothing.

### 4. Mixing strings and bytes

Text mode expects strings.

Binary mode expects bytes.

### 5. Forgetting newline characters with `write()` or `writelines()`

Python does not automatically add a new line unless you include `\n` yourself.

### 6. Not using `with open()`

Beginners sometimes use `open()` without closing files properly.

Using `with open()` is safer.

### 7. Counting characters incorrectly

Be clear whether newline characters should be counted.

`len(line)` includes them unless stripped first.

### Debugging tips

- print file content after writing to confirm results
- check whether you used the correct file mode
- use `seek(0)` when re-reading the same file object
- print `type(data)` when unsure whether you are handling strings or bytes

- test on small files first before working with large ones

---

## 7. Mentor tips / best practices

- always prefer `with open()` for file work
- be careful with `w` mode because it overwrites data
- use `a` mode for logs or updates
- use meaningful file names during practice
- separate reading logic from processing logic when programs get bigger
- use text mode for readable content and binary mode for raw data
- test file content after every major operation while learning

### Best-practice comparison

Instead of writing:

```
file = open("data.txt", "r")

content = file.read()

file.close()
```

prefer:

```
with open("data.txt", "r") as file:

    content = file.read()
```

This is cleaner and safer.

### Mentor insight

File handling is one of those topics where tiny mistakes can silently cause data loss.

That is why good Python developers build habits around safe modes, careful testing, and clean structure.

---

## 8. Practice tasks

1. Create a text file and write three lines into it.
2. Read the whole file and print it.
3. Read the same file line by line using a loop.
4. Append one extra line without removing the original content.
5. Count the number of lines, words, and characters.
6. Copy the content of one file into another file.
7. Use `writelines()` to store a list of strings.
8. Use `w+` mode and `seek(0)` in one example.
9. Write bytes to a binary file and read them back.
10. Explain in your own words when to use `r`, `w`, and `a`.

### Sample solutions

```
with open("practice.txt", "w") as file:

    file.write("First line\nSecond line\nThird line\n")


with open("practice.txt", "r") as file:

    print(file.read())


with open("practice.txt", "a") as file:

    file.write("Appended line\n")


with open("practice.txt", "r") as file:

    lines = file.readlines()
```

```
print("Lines:", len(lines))

print("Words:", sum(len(line.split()) for line in lines))

print("Characters:", sum(len(line) for line in lines))


with open("binary_demo.bin", "wb") as file:

    file.write(b"ABC123")
```

---

## 9. Revision notes

- `open()` is used to work with files in Python
- `with open()` is the safest and most readable approach
- `r` reads existing files
- `w` writes and overwrites existing content
- `a` appends without deleting older content
- `read()`, `readline()`, and `readlines()` are common reading methods
- `write()` and `writelines()` are common writing methods
- binary files use `rb` and `wb`
- `w+` supports both writing and reading
- `seek()` moves the file cursor
- file handling is essential for practical Python programming

# Chapter 19 — Working with File Paths Using the `os` Module

## 1. Chapter title

**Working with File Paths Using the `os` Module**

---

## 2. Concept explanation

When you work with files in Python, knowing only how to open and read a file is not enough.

You also need to understand **where** files are stored, how folders are created, how file paths are built, and how to check whether a path points to a file or a directory.

This is where the `os` module becomes very important.

The `os` module helps Python communicate with the operating system.

It allows you to:

- create directories
- list files and folders
- join path parts safely
- get absolute paths
- check whether a path exists
- check whether a path is a file or a directory

These skills are especially important in end-to-end projects, automation, file handling, and scripting.

### What is a file path?

A **file path** tells Python where a file or folder is located.

Example:

notes/chapter1.txt

This is a path.

There are two main types of paths:

- **relative path**
- **absolute path**

### Relative path

A relative path is based on the current working directory.

Example:

data/report.csv

This means Python will look for the `data` folder inside the current working directory.

### Absolute path

An absolute path gives the full location starting from the root of the system.

Example on Windows:

C:\Users\Student\Documents\report.txt

Example on Linux or macOS:

/home/student/report.txt

Absolute paths are more specific, but less portable across systems.

### Why `os.path.join()` matters

Many beginners try to build paths like this:

folder + "/" + filename

This can work sometimes, but it is not the best practice because different operating systems use different path separators.

Instead, use:

os.path.join(folder, filename)

This builds a path in an operating-system-safe way.

### Creating directories

The `os.mkdir()` function creates a new directory.

Example:

```
import os
os.mkdir("package")
```

This creates a folder named `package` in the current working directory.

### Listing files and directories

The `os.listdir()` function shows what is present inside a folder.

Example:

```
import os
print(os.listdir("."))
```

Here `"."` means the current directory.

### Checking existence

You should not assume a file or folder exists.

Use:

```
os.path.exists(path)
```

This returns `True` if the path exists and `False` otherwise.

### Checking file or directory

Use:

- `os.path.isfile(path)` to check if a path is a file
- `os.path.isdir(path)` to check if a path is a directory

These checks are important when your code should behave differently for files and folders.

### Getting the absolute path

The function `os.path.abspath(path)` converts a relative path into an absolute path.

This is very useful for debugging and understanding where Python is actually looking.

### Creating a file if it does not exist

You can first check whether a file exists, and then create it if needed.

This is a common and safe file-handling practice.

### Lecture points preserved and expanded

From your lecture notes, the key ideas are:

- `os.mkdir()` creates directories
- `os.listdir()` lists files and directories
- `os.path.join()` builds safe paths
- relative and absolute paths are different
- `os.path.abspath()` helps obtain full paths
- `os.path.exists()` checks if something exists
- `os.path.isfile()` and `os.path.isdir()` identify path types

- these operations are essential for practical projects

---

## 3. Why it matters

Working with paths matters because almost every real Python program eventually works with files.

You may need to:

- read input data
- save reports
- create folders automatically
- organize project outputs
- check whether required files are present
- build cross-platform scripts
- avoid file-not-found errors

Without path handling, file operations become fragile.

This chapter matters because it helps you write code that is:

- safer
- more portable
- easier to debug
- more professional

It also prepares you for:

- automation scripts
- data projects

- web development
- project structure management
- file uploads and downloads
- configuration-based programs

---

## 4. Code examples

### Example 1: Import the `os` module

```
import os

print("OS module imported successfully")
```

### Example 2: Create a directory

```
import os


folder_name = "package"


if not os.path.exists(folder_name):

    os.mkdir(folder_name)

    print("Folder created")

else:

    print("Folder already exists")
```

### Example 3: List files and directories in the current folder

```
import os

items = os.listdir(".")
```

```
print(items)
```

### Example 4: Join paths safely

```
import os

folder = "data"
filename = "report.txt"
full_path = os.path.join(folder, filename)

print(full_path)
```

### Example 5: Get an absolute path

```
import os

relative_path = "data/report.txt"
absolute_path = os.path.abspath(relative_path)

print("Relative path:", relative_path)
print("Absolute path:", absolute_path)
```

### Example 6: Check whether a path exists

```
import os

path = "data/report.txt"
```

```
if os.path.exists(path):
    print("Path exists")
else:
    print("Path does not exist")
```

### Example 7: Check whether a path is a file

```
import os

path = "sample.txt"
print(os.path.isfile(path))
```

### Example 8: Check whether a path is a directory

```
import os

path = "package"
print(os.path.isdir(path))
```

### Example 9: Create a file if it does not exist

```
import os

filename = "notes.txt"

if not os.path.exists(filename):
    with open(filename, "w") as file:
        file.write("This file was created because it did not exist.\n")
```

```
    print("File created")
else:
    print("File already exists")
```

### Example 10: Combine path checking with file creation

```
import os

folder = "project_data"
file_name = "info.txt"
folder_exists = os.path.exists(folder)

if not folder_exists:
    os.mkdir(folder)

full_path = os.path.join(folder, file_name)

if not os.path.exists(full_path):
    with open(full_path, "w") as file:
        file.write("Project information file\n")

print(full_path)
```

### Example 11: Print the current working directory

```
import os
print(os.getcwd())
```

### Example 12: Compare relative and absolute paths

```
import os

path = "example.txt"
print("Relative:", path)
print("Absolute:", os.path.abspath(path))
```

### Example 13: Explore a folder step by step

```
import os

folder = "."
for item in os.listdir(folder):
    item_path = os.path.join(folder, item)
    if os.path.isdir(item_path):
        print(item, "-> Directory")
    elif os.path.isfile(item_path):
        print(item, "-> File")
```

### Example 14: Safer path building for nested folders

```
import os

path = os.path.join("project", "data", "raw", "file.csv")
print(path)
```

---

## 5. Explanation of code

### Example 1 explanation

This confirms the `os` module is available and ready to use.

### Example 2 explanation

This teaches safe directory creation.

The code checks whether the folder already exists before creating it, which avoids an error.

### Example 3 explanation

This shows how to inspect the contents of the current directory.

It is useful when debugging file-related programs.

### Example 4 explanation

This demonstrates one of the most important path operations: building paths safely using `os.path.join()`.

### Example 5 explanation

This teaches the difference between relative and absolute paths through a direct example.

### Example 6 explanation

This helps prevent errors by checking whether a path exists before trying to use it.

### Example 7 explanation

This shows how to confirm that a path points to a file, not just to an existing item.

### Example 8 explanation

This is the directory version of the previous example.

### Example 9 explanation

This demonstrates a practical pattern: create a file only if it is missing.

### Example 10 explanation

This combines multiple ideas:

- create a folder if needed
- build a full path safely
- create a file inside that folder

This is very close to what real automation scripts often do.

### Example 11 explanation

This prints the current working directory, which is one of the first things to check when path-related bugs happen.

### Example 12 explanation

This gives a direct comparison between relative and absolute paths.

### Example 13 explanation

This teaches how to inspect items in a folder and classify them as files or directories.

### Example 14 explanation

This reinforces that `os.path.join()` is better than manually writing separators.

---

## 6. Common mistakes

### 1. Building paths manually with slashes

This is not ideal:

```
folder + "/" + filename
```

Use:

```
os.path.join(folder, filename)
```

### 2. Assuming the current working directory is what you expect

Sometimes your script runs from a different directory than you think.

Always check with:

```
os.getcwd()
```

### 3. Creating a directory that already exists

Calling `os.mkdir()` on an existing folder raises an error unless you check first.

### 4. Confusing relative and absolute paths

A relative path may fail if the current working directory changes.

### 5. Using `exists()` but not checking file type

A path may exist but still be the wrong type.

For example, you may expect a file but actually have a directory.

### 6. Forgetting nested directories

`os.mkdir()` creates only one directory level at a time.

For deeply nested structures, another approach is needed.

### Debugging tips

- print the result of `os.getcwd()`
- print full absolute paths while testing
- check `os.path.exists()` before opening files
- use `os.path.isfile()` and `os.path.isdir()` for clarity
- inspect folder contents with `os.listdir()`
- avoid guessing where your files are located

---

## 7. Mentor tips / best practices

- always use `os.path.join()` for building paths
- check existence before creating or opening files
- print absolute paths during debugging
- separate folder paths and file names clearly in your code
- keep file-handling code defensive and explicit
- use readable variable names like `folder_path`, `file_path`, and `base_dir`

### Best-practice example

```
import os


base_dir = "reports"

file_name = "summary.txt"

file_path = os.path.join(base_dir, file_name)
```

This is clearer and safer than manually mixing strings.

### Mentor insight

Path handling looks small at first, but it is a major skill in real projects.

Many beginner errors in Python are not about syntax — they are about the program looking in the wrong place.

A strong Python developer learns to treat paths carefully and systematically.

---

## 8. Practice tasks

11. Create a directory named `data_files`.

12. Print all items in the current folder.
13. Join `"reports"` and `"final.txt"` into one path.
14. Print the absolute path of `"notes.txt"`.
15. Check whether `"notes.txt"` exists.
16. Create a file only if it does not already exist.
17. Check whether `"data_files"` is a directory.
18. Check whether `"example.txt"` is a file.
19. Print the current working directory.
20. Loop through the current folder and print whether each item is a file or directory.

### Sample solutions

```
import os

if not os.path.exists("data_files"):

    os.mkdir("data_files")
```

```
import os

print(os.listdir("."))
```

```
import os

print(os.path.join("reports", "final.txt"))
```

```
import os

print(os.path.abspath("notes.txt"))
```

```
import os

print(os.path.exists("notes.txt"))
```

---

## 9. Revision notes

- the `os` module helps Python work with the operating system
- `os.mkdir()` creates directories
- `os.listdir()` lists folder contents
- `os.path.join()` builds safe paths
- relative paths depend on the current working directory
- absolute paths show the full location
- `os.path.abspath()` converts relative paths into absolute paths
- `os.path.exists()` checks if a path exists
- `os.path.isfile()` checks for files
- `os.path.isdir()` checks for directories
- strong file handling depends on strong path handling

# Chapter 20 — Exception Handling in Python

## 1. Chapter title

**Exception Handling in Python**

---

## 2. Concept explanation

When a Python program runs, sometimes something goes wrong.

For example:

- dividing by zero
- trying to open a file that does not exist
- converting invalid text into a number
- using a variable that has not been defined

These problems are called **exceptions**.

An **exception** is an event that interrupts the normal flow of a program.

If exceptions are not handled, the program may stop suddenly and show an error message.

But with **exception handling**, you can manage errors more gracefully and keep the program under control.

This is one of the most important skills in Python because real programs must deal with unexpected situations.

### Common exception types

Some common built-in exceptions are:

- `ZeroDivisionError`
- `FileNotFoundError`
- `ValueError`
- `NameError`
- `TypeError`
- `IndexError`
- `KeyError`

Each exception type represents a different kind of problem.

### Why exception handling matters

Without exception handling, even a small user mistake can crash a program.

For example:

- a user types `"abc"` instead of a number
- a file path is wrong
- a calculation uses invalid data
- external data is missing

A strong Python program does not just work when everything is perfect.

It also behaves properly when something goes wrong.

### The `try` block

The `try` block contains code that might raise an exception.

Example:

```
try:
    number = int(input("Enter a number: "))
    print(10 / number)
```

Python runs the code inside `try`.

If no error happens, execution continues normally.

### The `except` block

The `except` block runs if an exception occurs in the `try` block.

Example:

```
try:
    num = int("abc")
except ValueError:
    print("Please enter a valid number.")
```

Here, instead of crashing, the program shows a user-friendly message.

### The `else` block

The `else` block runs only if no exception happened in the `try` block.

This is useful when you want some code to execute only after successful completion.

### The `finally` block

The `finally` block runs whether an exception happened or not.

This is very useful for cleanup tasks such as:

- closing files
- closing database connections
- releasing resources
- printing final status messages

### Handling multiple exceptions

You can use multiple `except` blocks to handle different error types separately.

This gives more specific and helpful responses.

### Catch-all handling with `Exception`

All standard exceptions inherit from the base `Exception` class.

That means you can write:

```
except Exception as e:
    print("Something went wrong:", e)
```

This catches unexpected exceptions.

However, it should be used carefully.

It is usually better to handle known exceptions specifically first.

### Lecture points preserved and expanded

From your lecture notes, the key ideas are:

- exceptions disrupt normal program flow
- common examples include `ZeroDivisionError`, `FileNotFoundError`, `ValueError`, and `NameError`
- `try` is used for risky code
- `except` handles errors
- `else` runs when no exception occurs
- `finally` always runs
- multiple exceptions can be handled separately
- all exceptions come from the base `Exception` class
- practical uses include file handling and input validation
- good exception handling improves user experience and program reliability

---

## 3. Why it matters

Exception handling matters because real programs deal with real users and real data.

That means things often go wrong.

Users may:

- enter incorrect values
- provide missing files
- use invalid menu choices
- cause unexpected states in the program

Without exception handling, these situations can crash the program.

With exception handling, you can:

- prevent sudden crashes
- show meaningful messages
- guide users correctly
- keep programs stable
- separate normal logic from error-handling logic

This is especially important in:

- file operations
- user input
- data parsing
- web applications
- APIs
- automation scripts
- larger software systems

A beginner writes code that works.

A strong Python developer writes code that also fails safely.

---

## 4. Code examples

### Example 1: Basic `try` and `except`

```
try:
    result = 10 / 0
    print(result)
except ZeroDivisionError:
```

```
    print("You cannot divide by zero.")
```

### Example 2: Handling `ValueError`

```
try:
    age = int("twenty")
    print(age)
except ValueError:
    print("Invalid number format.")
```

### Example 3: Handling file errors

```
try:
    with open("missing_file.txt", "r") as file:
        content = file.read()
except FileNotFoundError:
    print("The file was not found.")
```

### Example 4: Using `else`

```
try:
    number = int("25")
except ValueError:
    print("Conversion failed.")
else:
    print("Conversion successful.")
    print("Number is:", number)
```

### Example 5: Using `finally`

```
try:
    result = 10 / 2
    print(result)
except ZeroDivisionError:
    print("Division error.")
finally:
    print("This block always runs.")
```

### Example 6: Multiple `except` blocks

```
try:
    value = int("hello")
    result = 10 / value
except ValueError:
    print("Please enter a valid integer.")
except ZeroDivisionError:
    print("Division by zero is not allowed.")
```

### Example 7: Catching all exceptions

```
try:
    data = {"name": "Asha"}
    print(data["age"])
except Exception as e:
    print("An unexpected error occurred:", e)
```

### Example 8: Handling `NameError`

```
try:
    print(total_amount)
except NameError:
    print("The variable is not defined.")
```

### Example 9: Safe user input conversion

```
user_input = "42"

try:
    number = int(user_input)
    print("You entered:", number)
except ValueError:
    print("Please enter a valid integer.")
```

### Example 10: File handling with `finally`

```
file = None

try:
    file = open("sample_exception.txt", "w")
    file.write("Testing exception handling.")
except Exception as e:
    print("Error:", e)
finally:
    if file:
```

```
    file.close()
    print("File closed safely.")
```

### Example 11: Using `else` and `finally` together

```
try:
    x = 10
    y = 5
    result = x / y
except ZeroDivisionError:
    print("Cannot divide by zero.")
else:
    print("Result:", result)
finally:
    print("Calculation complete.")
```

### Example 12: Checking list access

```
try:
    numbers = [1, 2, 3]
    print(numbers[5])
except IndexError:
    print("List index is out of range.")
```

### Example 13: Handling dictionary key errors

```
try:
    student = {"name": "Ravi"}
```

```
    print(student["marks"])
except KeyError:
    print("Key not found in dictionary.")
```

### Example 14: Simple validation function

```
def divide_numbers(a, b):
    try:
        return a / b
    except ZeroDivisionError:
        return "Cannot divide by zero."

print(divide_numbers(10, 2))
print(divide_numbers(10, 0))
```

### Example 15: Broad catch after specific catches

```
try:
    value = int("abc")
except ValueError:
    print("ValueError handled.")
except Exception as e:
    print("Other exception:", e)
```

---

## 5. Explanation of code

### Example 1 explanation

This introduces the most basic form of exception handling.

The program attempts a risky operation and catches a specific error.

### Example 2 explanation

This shows how conversion errors happen when invalid text is converted into an integer.

### Example 3 explanation

This is a real-world example.

File-related code often needs exception handling because files may be missing or paths may be wrong.

### Example 4 explanation

This introduces the `else` block, which runs only when the `try` block succeeds.

### Example 5 explanation

This shows `finally`, which always runs whether an exception occurred or not.

### Example 6 explanation

This teaches how to handle different exceptions separately for clearer error messages.

### Example 7 explanation

This shows a catch-all approach using `Exception as e`.

It is useful for debugging or unexpected errors, but should not replace all specific handling.

### Example 8 explanation

This demonstrates a `NameError`, which happens when you use a variable that does not exist.

### Example 9 explanation

This is a practical user-input pattern where invalid conversion is handled safely.

### Example 10 explanation

This teaches cleanup logic.

Even if something goes wrong, `finally` ensures the file gets closed properly.

### Example 11 explanation

This combines `except`, `else`, and `finally` in one clear flow.

### Example 12 explanation

This shows how to handle `IndexError` when accessing a list outside its valid range.

### Example 13 explanation

This shows `KeyError`, which happens when a dictionary key is missing.

### Example 14 explanation

This demonstrates exception handling inside a function, which is common in reusable code.

### Example 15 explanation

This teaches the correct pattern of handling specific exceptions before a general catch-all block.

---

## 6. Common mistakes

### 1. Using a broad `except` too early

This is not a good habit:

```
except:
    print("Error")
```

It hides useful information and can make debugging harder.

### 2. Catching `Exception` before specific exceptions

Wrong order:

```
except Exception:
    ...
except ValueError:
    ...
```

The general exception should come after the specific ones.

### 3. Using exception handling for normal control flow

Exceptions should handle exceptional cases, not replace ordinary checks.

### 4. Writing unclear error messages

Error messages should help users understand what went wrong.

### 5. Forgetting cleanup tasks

Files, connections, or resources may stay open if you do not clean them up properly.

### 6. Catching errors but ignoring them completely

Silent failures make debugging difficult.

### Debugging tips

- print the exception object with `as e`
- catch specific exceptions whenever possible
- use `finally` for cleanup
- test both success and failure cases
- avoid hiding useful errors with overly broad handlers
- make error messages meaningful and human-readable

---

## 7. Mentor tips / best practices

- handle the exceptions you truly expect
- prefer specific exception types over broad catches
- use `else` for code that should run only on success
- use `finally` for cleanup work
- write messages that help both users and developers
- combine validation and exception handling carefully
- keep error-handling code readable, not cluttered

### Best-practice example

```
try:
    number = int(user_input)
except ValueError:
    print("Please enter a valid number.")
else:
    print("Valid number:", number)
```

This is clearer than mixing success logic and error handling in the same block.

### Mentor insight

Exception handling is not about hiding errors.

It is about responding to errors wisely.

A well-handled exception makes a program feel reliable and professional.

---

## 8. Practice tasks

21. Write a program that catches `ZeroDivisionError`.

22. Convert user text to an integer and handle `ValueError`.
23. Try opening a missing file and catch `FileNotFoundError`.
24. Use `else` in a successful conversion example.
25. Use `finally` to print a closing message.
26. Handle both `ValueError` and `ZeroDivisionError` in one program.
27. Catch a `NameError` example.
28. Catch an `IndexError` when accessing a list.
29. Catch a `KeyError` when reading a dictionary key.
30. Write a small function that uses exception handling internally.

## Sample solutions

```
try:
    print(10 / 0)
except ZeroDivisionError:
    print("Cannot divide by zero.")


try:
    value = int("abc")
except ValueError:
    print("Please enter a valid integer.")


try:
    with open("missing.txt", "r") as file:
        print(file.read())
except FileNotFoundError:
    print("File does not exist.")


try:
    num = int("50")
except ValueError:
```

```
    print("Invalid input.")
else:
    print("Converted:", num)


try:
    print("Running...")
finally:
    print("Always executed.")
```

---

## 9. Revision notes

- an exception interrupts the normal flow of a program
- common exceptions include `ZeroDivisionError`, `ValueError`, `FileNotFoundError`, and `NameError`
- `try` contains risky code
- `except` handles errors
- `else` runs only if no exception occurs
- `finally` always runs
- specific exceptions should usually be handled before general ones
- `Exception` is the parent class of standard exceptions
- exception handling improves program stability and user experience
- strong Python programs do not just work — they fail gracefully too